JobBoard — Dossier d'Architecture Logicielle
Projet : Plateforme de recrutement en microservices

Sommaire

1. Contexte et Problématique
2. Analyse Fonctionnelle (Acteurs et Cas d'Utilisation)
3. Découpage en Microservices (Architecture Métier)
4. Modèle de Données et Stockage
5. Interactions et Communications
6. Architecture Technique Globale
7. Sécurité
8. Organisation du Dépôt Git et Gestion de Projet
9. Planning et Gestion des Risques
10. Conclusion
11. Contexte et Problématique

1.1 Contexte du Projet

La plateforme JobBoard est une application de recrutement en ligne mettant en relation des candidats et des recruteurs. Développée initialement sous la forme d'un monolithe (Symfony 6.4, base de données unique, déploiement monolithique), l'application fait face à de nouveaux défis. La direction souhaite ouvrir la plateforme aux terminaux mobiles et desktop, tout en garantissant une scalabilité lors des campagnes de recrutement intenses, et en permettant des livraisons continues sans interruption de service.

1.2 Problématique

L'architecture monolithique actuelle limite ces évolutions : le couplage fort entre les modules empêche toute modification isolée, le déploiement nécessite l'arrêt total de la plateforme, et l'absence d'API bloque l'intégration de nouveaux terminaux.

Problématique retenue : Comment découper JobBoard en microservices autonomes, sécurisés et communicants, tout en garantissant la cohérence métier des candidatures, pour permettre le multi-terminal, la scalabilité et le déploiement indépendant ?

2. Analyse Fonctionnelle

2.1 Les Acteurs

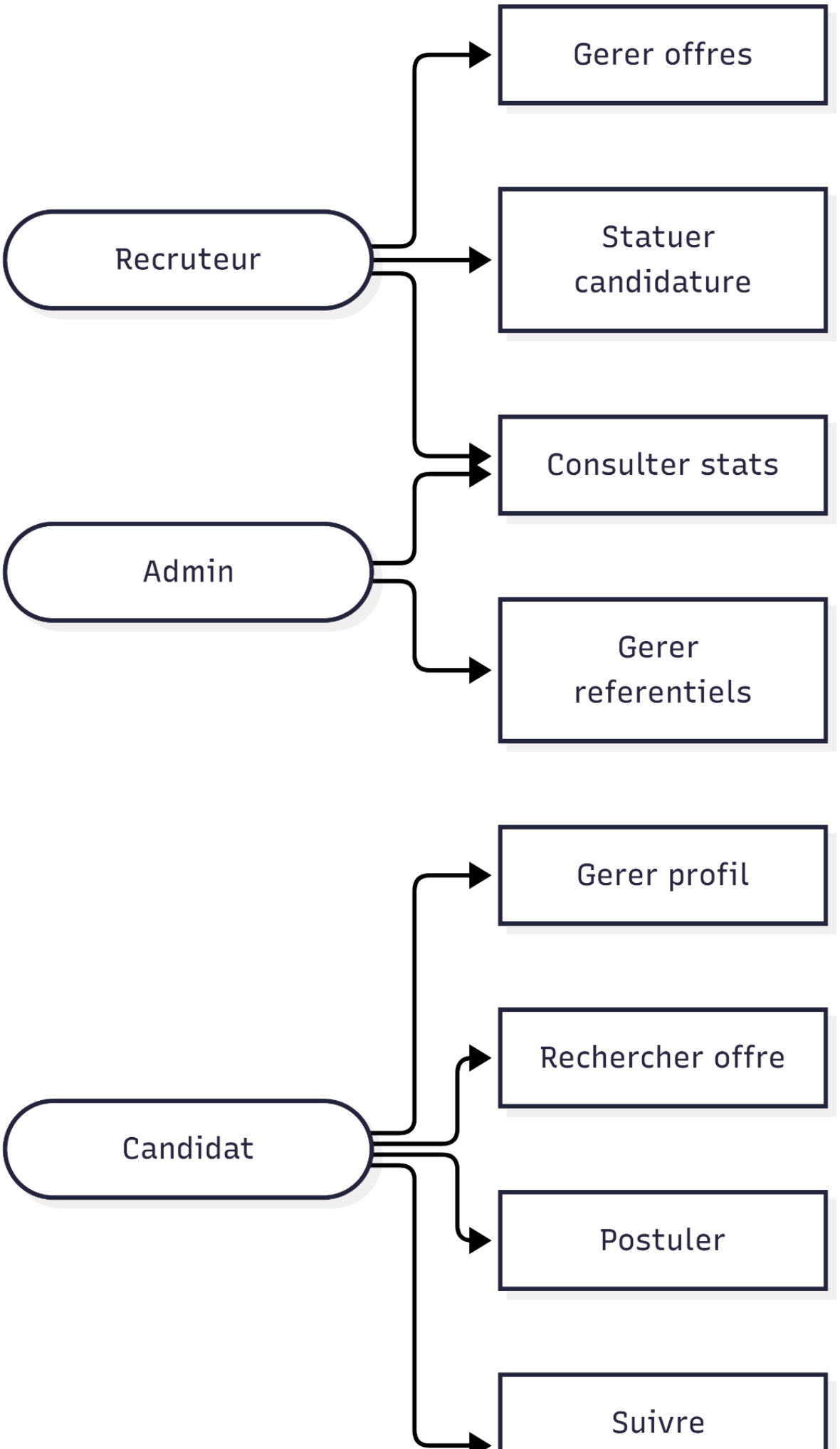
Le système interagit avec quatre profils principaux :

- Candidat : Crée son profil, recherche des offres, postule et suit l'état de ses candidatures.
- Recruteur : Publie des offres, consulte les profils des postulants et statue sur les candidatures.
- Administrateur : Gère les référentiels (catégories, compétences), modère la plateforme et consulte les statistiques globales.
- Système : Gère les processus automatisés (notifications, calcul des statistiques, journalisation).
-

2.2 Cas d'Utilisation Principaux

- UC1 : Gérer son profil candidat.
- UC2 : Rechercher une offre d'emploi.
- UC3 : Postuler à une offre.
- UC4 : Consulter et changer le statut d'une candidature.
- UC5 : Gérer les offres d'emploi (création, clôture).
- UC6 : Recevoir des notifications (e-mail/SMS).
- UC7 : Gérer les référentiels (catégories, localisations).

- UC8 : Consulter les tableaux de bord statistiques.
  [Insérer ici l'image : 2-cas-utilisation.png]


Recruteur
Admin
Candidat
Gerer offres
Statuer candidature
Consulter stats
Gerer referentiels
Gerer profil
Rechercher offre
Postuler
Suivre

3. Découpage en Microservices

Le découpage a été réalisé en suivant le principe de Contexte Borné (Bounded Context). Chaque service possède son propre langage métier, ses propres règles et sa propre base de données. Aucune clé étrangère n'est partagée entre les bases ; les services communiquent par identifiants externes (API) ou par événements.

L'architecture repose sur 6 microservices :

1. candidate-service : Gère l'identité et le profil des candidats. Domaine stable, indépendant du cycle de vie des offres.
2. job-service : Gère le cycle de vie des offres d'emploi (publication, modification, clôture).
3. application-service : Cœur du système. Gère la jonction métier entre un candidat et une offre (la candidature). Il orchestre les vérifications.
4. notification-service : Service purement réactif (event-driven). Il écoute les événements métier et génère les communications (e-mails, alertes).
5. analytics-service : Service avancé. Il agrège les événements de la plateforme pour générer des statistiques (utilise MongoDB pour sa flexibilité).
6. common-service : Service commun. Gère les référentiels transverses (catégories, compétences, villes) afin d'éviter la duplication de données dans les autres services.

7. Modèle de Données et Stockage

Le principe fondamental est "Une base de données par microservice" (Database per Service). Il n'y a aucune base de données partagée, garantissant le découplage et l'autonomie des équipes.

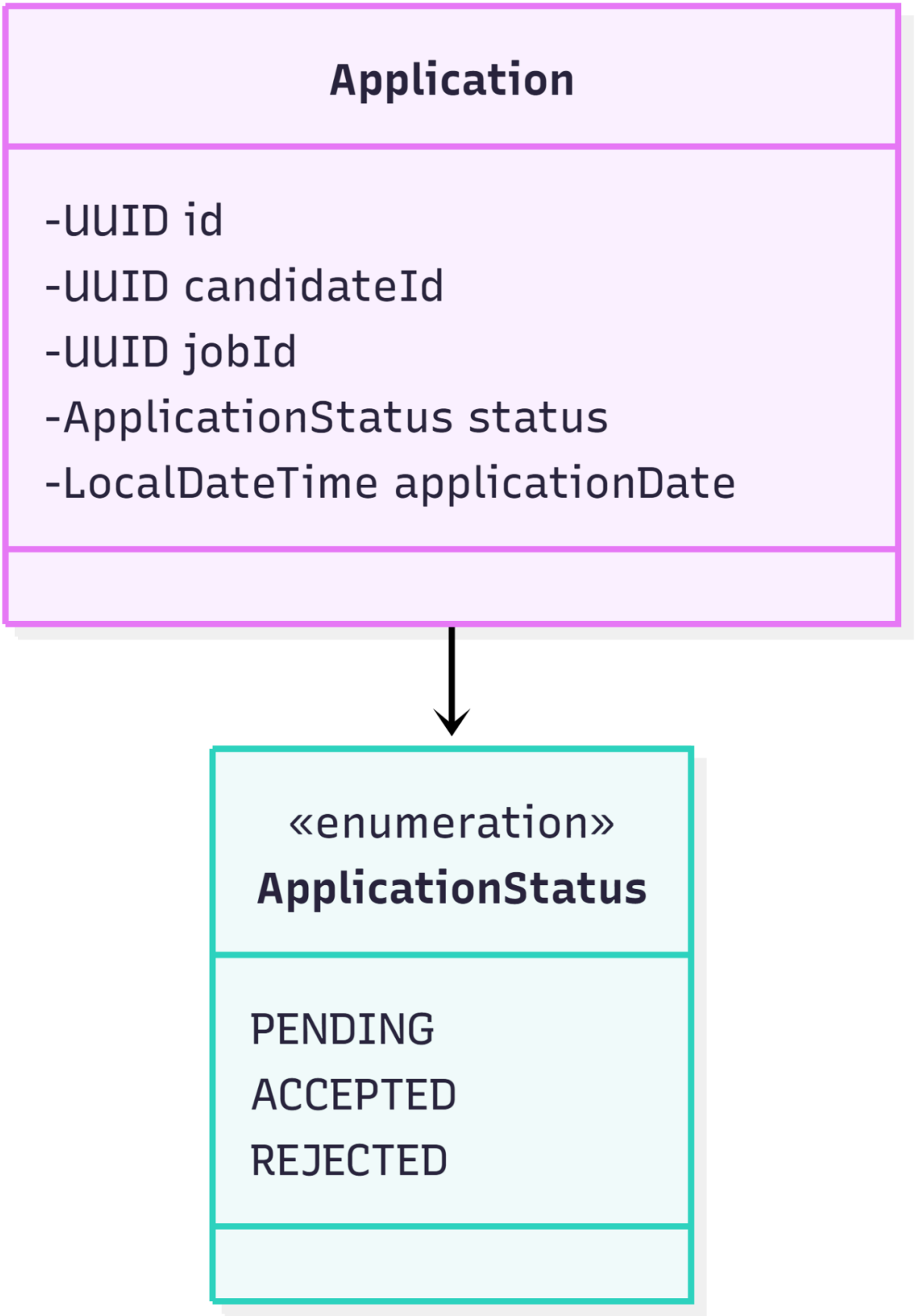
4.1 Choix Technologiques

- PostgreSQL (x5) : Utilisé pour les services candidate, job, application, notification et common. Ce choix est justifié par le besoin de transactions ACID (surtout pour les candidatures) et l'utilisation de contraintes d'intégrité fortes (ex: unicité d'une candidature par candidat/offre).
- MongoDB (x1) : Utilisé pour analytics-service. Ce choix est justifié par la nature hétérogène des événements collectés, le fort volume d'écriture (write-heavy) et le besoin d'agrégations flexibles via des pipelines.
-

4.2 Aperçu des Entités

- candidate_db : Candidate (id, email UNIQUE, nom), Address.
- job_db : Job (id, title, categoryId, status), JobSkill.
- application_db : Application (id, candidateId, jobId, status). Contrainte : UNIQUE(candidateId, jobId).
- notification_db : Notification (id, recipient, status, templateId), Template.
- common_db : Category, Skill, Location.

- analytics_db : Collection events (payload flexible), Collection daily_metrics.
  [Insérer ici l'image : 5-classes-application.png]



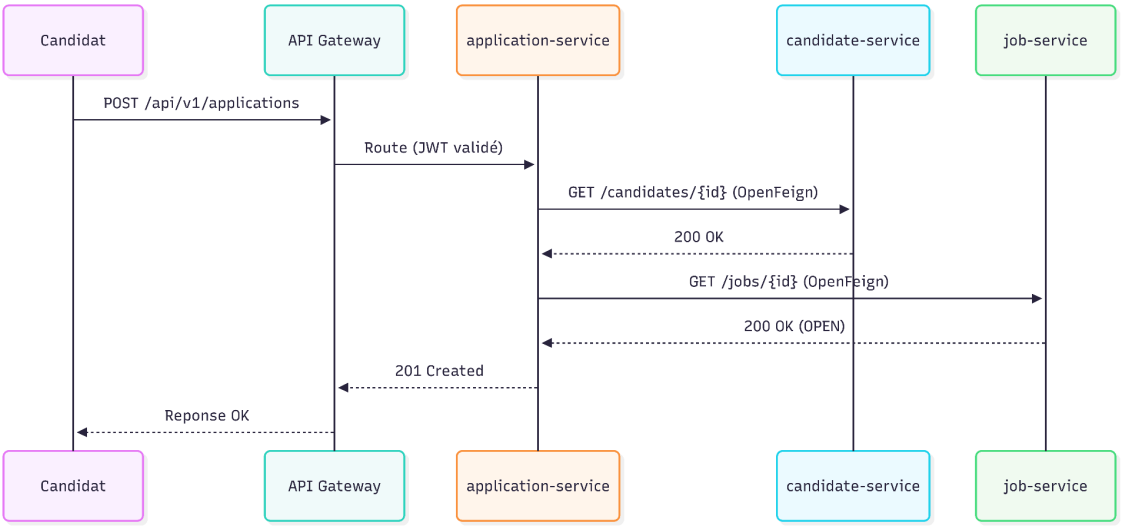
5. Interactions et Communications

Les services communiquent selon deux modes : synchrone (pour les vérifications immédiates) et asynchrone (pour la propagation d'événements).

5.1 Communication Synchrone (OpenFeign)

Utilisée uniquement lorsque le service a besoin d'une réponse immédiate pour valider une opération métier.

- Exemple : Lors d'une candidature, application-service appelle candidate-service pour vérifier l'existence du candidat, et job-service pour vérifier que l'offre est ouverte. Si un service est indisponible, application-service renvoie une erreur 503 Service Unavailable (gérée via un Circuit Breaker Resilience4j).
  [Insérer ici l'image : 3-sequence-sync-candidature.png]

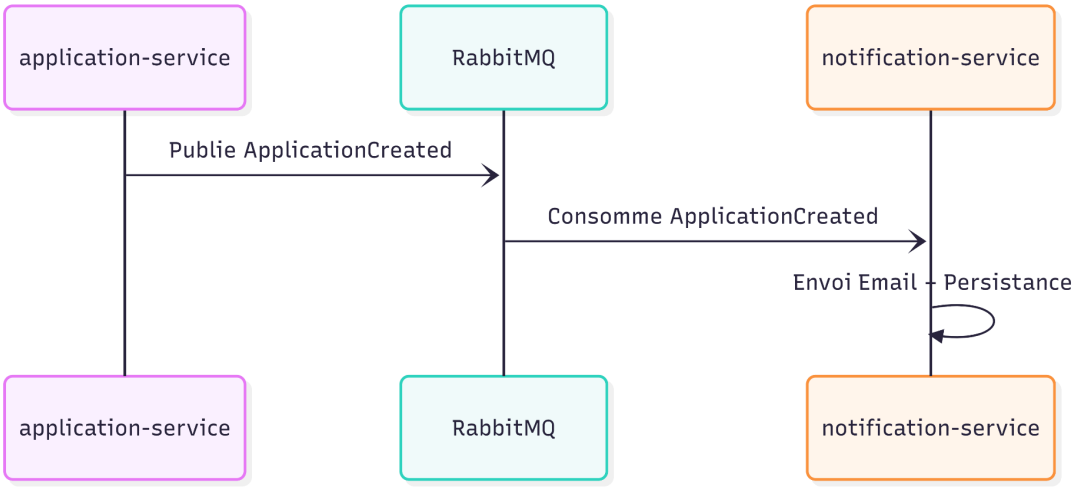


5.2 Communication Asynchrone (RabbitMQ)

Utilisée pour les événements métier (Event-Driven). Les services publient des événements sur un broker RabbitMQ (Exchange Topic) sans attendre de réponse.

- Événements principaux : ApplicationCreated, ApplicationStatusChanged, JobPublished.
- Consommateurs : notification-service (pour envoyer les e-mails) et analytics-service (pour les statistiques).
- Gestion des pannes : En cas d'échec de traitement par un consommateur, le message est retenté (Retry) puis envoyé dans une file d'attente de messages morts (Dead Letter Queue - DLQ).

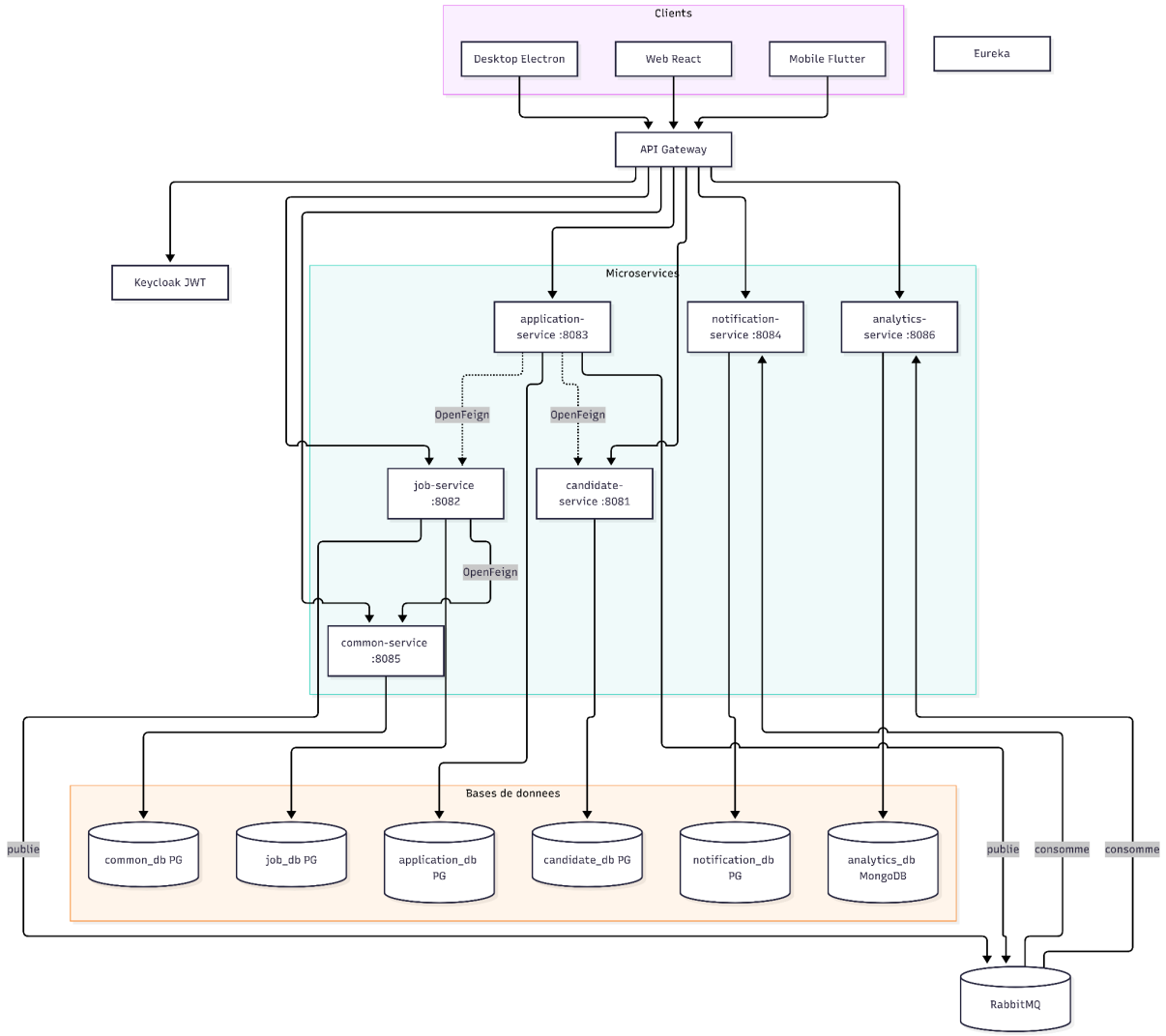
[Insérer ici l'image : 4-sequence-async-notification.png]



6. Architecture Technique Globale

L'architecture technique s'appuie sur l'écosystème Spring Cloud pour répondre aux problématiques des systèmes distribués.

[Insérer ici l'image : 1-architecture-globale.png]



Composants clés :

- API Gateway : Point d'entrée unique pour les clients (Web, Mobile, Desktop). Il gère le routage, le rate-limiting et délègue l'authentification.
- Eureka (Service Discovery) : Annuaire permettant aux microservices de se localiser dynamiquement par leur nom logique.
- Config Server : Serveur de configuration centralisé pour gérer les propriétés par environnement.
- Keycloak : Gestionnaire d'identité (IAM) fournissant des jetons JWT.
- Docker Compose : Utilisé pour l'orchestration locale de l'ensemble de l'infrastructure (bases de données, broker, microservices).

7. Sécurité

La sécurité est centralisée et déléguée à Keycloak (basé sur les standards OAuth2 et OpenID Connect).

- Les clients s'authentifient auprès de Keycloak et obtiennent un JWT (JSON Web Token).
- L'API Gateway intercepte les requêtes, valide la signature du JWT et injecte les rôles de l'utilisateur (CANDIDATE, RECRUITER, ADMIN) dans les en-têtes de la requête.
- Chaque microservice agit en tant que Resource Server (Spring Security) : il vérifie les autorisations basées sur les rôles avant d'exécuter une action (ex: seul un RECRUITER peut publier une offre).

8. Organisation du Dépôt Git et Gestion de Projet

8.1 Structure du Dépôt

Le dépôt Git respecte une structure monorepo stricte :

- /Backend : Contient les 6 projets Spring Boot indépendants.
- /Frontend : Contient le code de l'application cliente (React/Angular).
- /Documentation : Contient les diagrammes (/Diagrams), le dossier technique (/Technique) et la présentation (/Presentation).
- docker-compose.yml : Fichier d'orchestration à la racine.

8.2 Gestion de Projet

- Branches : La branche main est protégée. Chaque fonctionnalité est développée sur une branche feat/<nom-du-service> puis fusionnée via Pull Request.
- Commits : Convention de nommage stricte (feat, fix, docs, chore) pour garantir un historique lisible.
- Suivi : Utilisation d'un tableau Kanban (GitHub Projects) pour suivre l'avancement des tâches.

9. Planning et Gestion des Risques

9.1 Planning Prévisionnel

- Semaine 1 : Conception, validation de l'architecture, initialisation du dépôt.
- Semaine 2 : Création des squelettes, configuration des bases de données et de l'infrastructure Docker.
- Semaine 3 : Développement des API métier, implémentation des communications (Feign/RabbitMQ).
- Semaine 4 : Finalisation, tests d'intégration, rédaction de la documentation et préparation de la soutenance.

9.2 Gestion des Risques

- Risque : Indisponibilité d'un microservice lors d'un appel synchrone.
    - Mitigation : Implémentation de Resilience4j (Circuit Breaker et Timeouts) pour éviter les défaillances en cascade.
- Risque : Perte d'événements asynchrones (ex: RabbitMQ tombe).
    - Mitigation : Utilisation du pattern Outbox dans application-service (les événements sont d'abord sauvegardés en base de données puis publiés) et configuration des Dead Letter Queues.

10. Conclusion

L'architecture proposée marque une transition réussie du monolithe vers une architecture distribuée orientée services. Le découpage métier garantit l'autonomie des équipes et la scalabilité de la plateforme. L'utilisation conjointe de communications synchrones (pour la cohérence métier immédiate) et asynchrones (pour la résilience et la performance) permet de répondre aux exigences de la plateforme JobBoard. Cette phase de conception valide les fondations techniques nécessaires au démarrage du développement.